LET'S LEARN PYTHON

by Aditya Varma

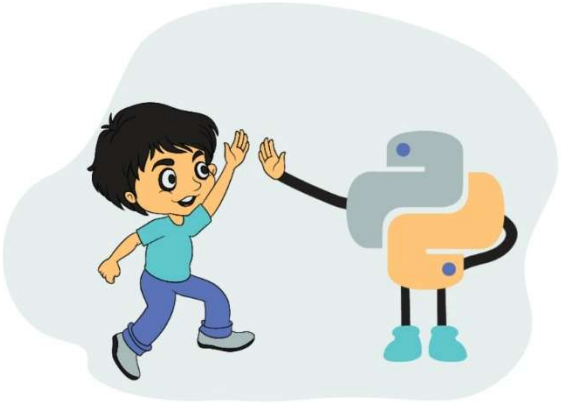
Index

Introduction

//What is Python?

Created by Guido van Rossum, Python is an object-oriented, general-purpose, high-level programming language.

The main aim of Python was to make programming easier for beginners and allow developers to get things done in fewer lines of code compared to other programming languages. Because of this simplicity, Python has become one of the most popular choices for learning programming in recent times.



Python is:

- General-purpose → can be used in web development, data science, AI, automation, and more.
- Interpreted → code runs line by line, no need for compilation.
- Interactive → you can test code directly in its shell (REPL).
- Object-Oriented → supports classes and objects for structured programming.
- Readable → uses English keywords and simple syntax rather than complex symbols.

Python is a perfect choice of programming language if you are a beginner. If you have any previous programming experience, then learning Python would be a cakewalk.

//History of Python:

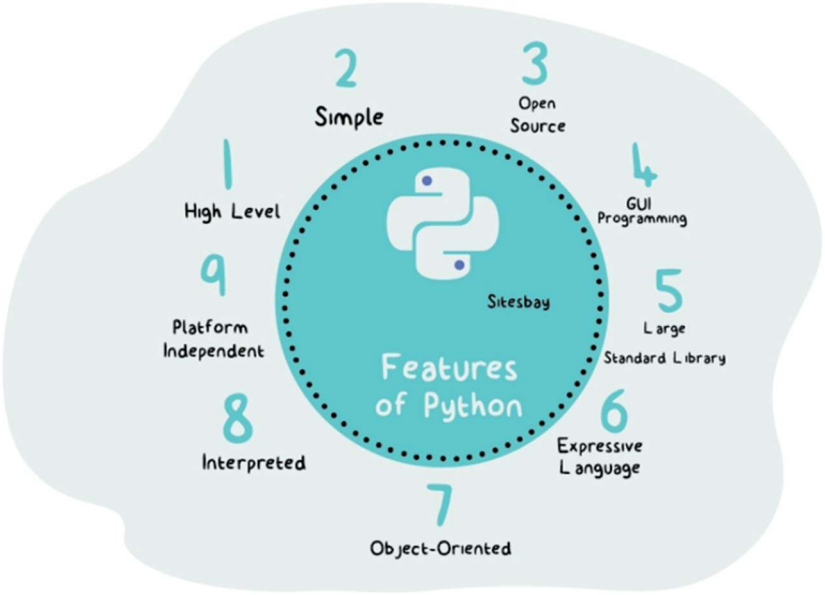
- 1985–1990: Guido van Rossum started working on Python at CWI (Centrum Wiskunde & Informatica) in the Netherlands.
- Inspiration: Python was designed as a successor to the ABC programming language. ABC had good features like exception handling and worked with the Amoeba operating system, but it also had some limitations.

- Guido wanted to keep the good parts of ABC while fixing its flaws. This led him to create a new scripting language that was simple, powerful, and practical.
- 1991: Python was officially released. The first version already included important features such as functions, exception handling, and core data types like strings, lists, and dictionaries.
- The Name "Python":
  The name did not come from the snake . Guido was a big fan of the BBC comedy show "Monty Python's Flying Circus". He wanted a name that was short, unique, and slightly mysterious, so he chose Python.

And that's how one of the world's most loved programming languages was born. Amazing, right?

//Features of Python:

- Easy-to-learn - Python has few keywords, simple structure, and a clearly defined syntax. This makes it easy to catch for beginners.
- Easy-to-read - Python code is more clearly defined and visible to the eyes.
- Easy-to-maintain - Python's source code is fairly easy-to-maintain
- Broad standard library - Python's bulk libraries are very portable and cross-platform compatible with UNIX, Windows, and Macintosh.
- Interactive Mode - Python has support for an interactive mode that allows interactive testing and debugging of snippets of code.
- Portable - Python can run on a wide variety of hardware platforms and has the same interface on all of those.
- Extendable - You can add low-level modules to the Python interpreter. These modules enable programmers to add or customize their tools to be more efficient



//Why Learn Python?

Do you remember that Python is a general-purpose programming language?
That means it can be used almost everywhere — and this is exactly why developers around the world love Python so much.

Python is used for:

- Web Development
- App Development
- Data Science
- Machine Learning
- Internet of Things (IoT)

...and the list goes on!

Let's deep-dive into some of these areas:

Web Development

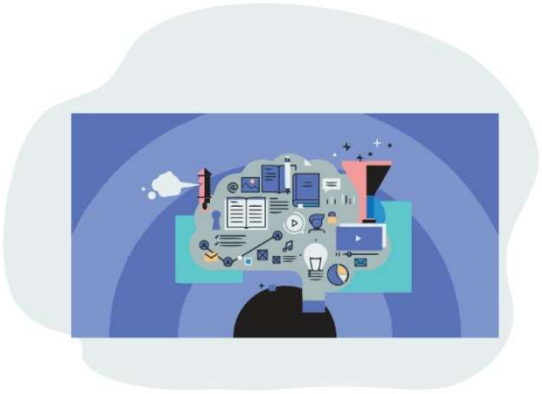
Python is one of the top choices for backend web development (not frontend).
Django → A powerful framework used for building large, full-fledged applications.
Flask → A lightweight framework suitable for smaller applications.
Both frameworks are in high demand, and developers skilled in Django or Flask often get high-paying roles.

App Development

Python can also be used for mobile applications.
While not the most popular choice for mobile apps, Python has a framework called Kivy, which allows you to build cross-platform apps (apps that run on both Android and iOS).

Data Science

One of the biggest reasons for Python's popularity today is its dominance in Data Science.
Python offers libraries for data processing, analysis, and visualization.
It has become the go-to language for data scientists because of its simplicity + powerful tools.
Skills in Python + Data Science are in very high demand in today's industry.



Machine Learning

Machine Learning might sound complicated, but Python makes it accessible even for beginners.
With libraries like Keras and Scikit-learn, you can build powerful ML models without diving deep into complex math and algorithms.
This makes Python a top choice for anyone who wants to step into AI and ML.

//Interpreted Python & Its Versions:

Of course, we know that Python is a programming language. But did you know there are two main types of programming languages?

1. Compiled Language
2. Interpreted Language

Chances of Error

- Compiled Languages:

  In a compiled language, your program is first converted into machine code before it runs. If there are any errors in your code, the compiler won't let the program execute until all errors are fixed. This means you see all errors at once before running the program.

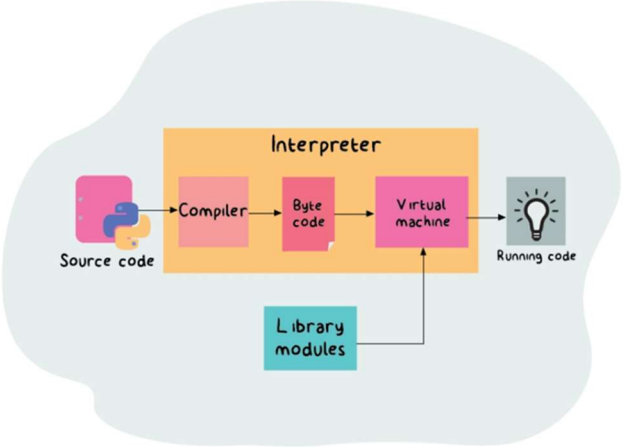
- Interpreted Languages:

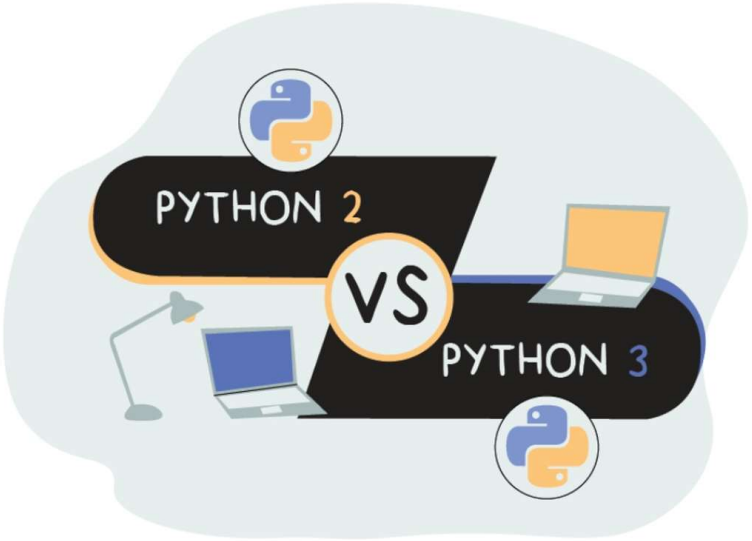
  An interpreted language starts running the code immediately, even if there are errors in parts of the program. When an error is encountered, the program stops execution, and an error message shows the problem. This allows you to test code line by line.

Python is an Interpreted Language

Python is a classic example of an interpreted programming language.

- Most Python implementations execute instructions directly without compiling the entire program into machine code.
- The interpreter translates each statement into subroutines and then into machine code as the program runs.
- This makes Python flexible, interactive, and beginner-friendly, because you can write and test code quickly.





Python Versions

- Python 2 → Legacy Version
  - Python 2 was popular for over a decade, and some companies still use it.
  - However, it is slowly becoming obsolete.
- Python 3 → Current and Future Version
  - Python 3 fixes many of Python 2's limitations and is actively supported.
  - For beginners, it is recommended to learn Python 3, as most modern development uses it.

//Installing Python & Setting up IDEs

Installing Python:

Before you start coding, you need to have Python installed on your computer. Follow these steps:

Step 1: Check if python is installed

1. Open **Command Prompt (Windows)** or **Terminal (Mac/Linux)**.
2. Type:

3. You should see the installed Python version
4. If not then follow the below steps

Step 2: Download Python

1. Go to the official website: https://www.python.org/downloads/
2. Click **Download Python 3.x** (latest stable version).

Step 3: Install Python

1. Run the downloaded installer.
2. **Important:** Check the box **"Add Python to PATH"** before clicking **Install Now**.
   - This makes Python accessible from the command line.
3. Follow the installation prompts.

Step 4: Verify Installation

1. Open **Command Prompt (Windows)** or **Terminal (Mac/Linux)**.
2. Type:

3. You should see the installed Python version, e.g., Python 3.12.0.

Python is now ready to run programs!

Installing PyCharm IDE:

PyCharm is a popular Python IDE (Integrated Development Environment) that makes coding easier for beginners and professionals.

Step 1: Download PyCharm

1. Go to https://www.jetbrains.com/pycharm/download/
2. Choose Community Edition (free) and download it.

Step 2: Install PyCharm

1. Run the installer.
2. Select installation location (default is fine).
3. Check "Add launchers to PATH" if available.

4. Finish installation.

Step 3: Launch PyCharm and Configure

1. Open PyCharm.
2. On first launch, you can choose the UI theme (light or dark).
3. Create a new project:
    - Go to File → New Project
    - Choose Python Interpreter → Select the installed Python version
    - Click Create

Writing Your First Python Program in PyCharm:

1. Right-click your project folder → **New** → **Python File**
2. Name the file hello.py
3. Type the following code:

```
1    print("Hello, World!")
```

4. Right-click inside the code editor → **Run 'hello'**
5. You should see the output:

```
Hello, World!

Process finished with exit code 0
```

Congratulations! You have successfully set up Python and PyCharm and run your first program.

//Writing and Running Python Programs

Once Python is installed, you can start writing and running programs in multiple ways — no IDE required.

Using Python Interactive Shell (REPL):

Python comes with an **interactive mode**, also called the **REPL** (Read-Evaluate-Print Loop). It's great for experimenting and testing small pieces of code.

How to open Python REPL:

- Windows: Open Command Prompt → type python → press Enter
- Mac/Linux: Open Terminal → type python3 → press Enter

You should see something like:

```
Python 3.x.x (default, ... )
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Example:

```
>>> print("Hello, Python!")
Hello, Python!
>>> 5 + 7
12
```

The >>> is called the **Python prompt**, where you can type commands directly.

Using a Python Script File:

For longer programs, you usually write your code in a **Python script file** (.py) and then run it.

Steps:

1. Open any text editor (Notepad, VS Code, etc.).
2. Write your Python code, for example:

```
print("Hello, Python!")
x = 5
y = 10
print("Sum =", x + y)
```

3. Save the file with a **.py extension**, e.g., program.py.
4. Run the program:
    - **Windows:** Open Command Prompt → navigate to the file's folder → type python program.py → Enter
    - **Mac/Linux:** Open Terminal → navigate to the folder → type python3 program.py → Enter

You should see output like:

```
Hello, Python!
Sum = 15
```

Key Points for Beginners

- Python programs can be written and run without an IDE.
- Interactive Shell is ideal for small tests and learning concepts.
- Python script files are used for larger programs and permanent code.
- Always save your files with .py extension.

Lesson Program: First program in python.

Source Code:

greetings.py

```
userName = input("Enter your name: ")
userAge = int(input("Enter your age: "))
userHeight = float(input("Enter your height in meters: "))

print("\nSummary:")
print("Name:", userName)
print("Age:", userAge, "years")
print("Height:", userHeight, "meters")
```

PYTHON BASICS

//Python Syntax & Indentation

What is Syntax?

Every programming language has a set of rules that define how you should write code so that the computer can understand it. These rules are called the syntax of the language.

In Python, syntax is designed to be simple, clean, and readable, which is why beginners find it very easy to learn. Python code looks almost like plain English!

Example:

```
print("Hello, World!")
```

This is a valid Python statement. It prints a message to the screen. Unlike some other languages (like C, C++ or Java), Python doesn't need semicolons (;) at the end of statements — though you can use them if you want to put multiple statements on one line.

Statements and Lines:

Each line of code in Python is usually one statement.

Example:

```
x = 10
y = 20
print(x + y)
```

However, you can write multiple statements on a single line using semicolons:

```
x = 10; y = 20; print(x + y)
```

But this is not recommended for readability.

Indentation in Python:

In most programming languages, blocks of code (like inside an if, for, or function) are grouped using curly braces {}. But Python does it differently! Python uses indentation (spaces at the beginning of a line) to define code blocks.

For example:

```
if age >= 18:
    print("You are an adult.")
    print("You can vote.")
else:
    print("You are a minor.")
```

Here, the lines under the if and else statements are indented with 4 spaces, meaning they belong to those blocks. If you forget indentation or mix tabs and spaces, Python will show an 'IndentationError'.

Nested Indentation:

When you write blocks inside blocks, you increase the indentation level:

```
number = 10
if number > 0:
    print("Number is positive")
    if number % 2 == 0:
        print("It is also even")
    else:
        print("It is odd")
else:
    print("Number is negative")
```

Each inner block is indented more than the outer one.
This makes your code structure clear and readable.

Best Practices (PEP 8 Style Guide):

- Use 4 spaces per indentation level (no tabs).
- Be consistent throughout your code.
- Python will not tolerate inconsistent indentation!

Line Continuation:

Sometimes, your statement is too long to fit in one line. You can use a backslash (\) to continue it on the next line:

```
result = 100 + 200 + 300 + \
         400 + 500
```

You can also use parentheses ( ), brackets [ ], or braces { } for automatic continuation:

```
numbers = [
    1, 2, 3,
    4, 5, 6
]
```

Whitespace in Python:

Extra spaces around operators or inside parentheses don't affect execution.

```
value = ( 1 + 2 )
print(value)
```

But unnecessary spaces can make code look messy — so keep it neat!

Empty Blocks and the pass Statement

Python does not allow **empty code blocks**. If you need a placeholder for future code, use the pass statement.
Example:

```
if True:
    pass  # does nothing, avoids syntax error
```

//Comments in Python:

What are Comments?

Comments are notes written inside a program that are ignored by the Python interpreter. They are used to explain code, make it readable, and help others (or your future self) understand what the program does.

When Python runs your code, it skips all comments completely — they are meant only for humans, not computers.

Why Use Comments?

Comments make your code self-explanatory. They are useful for:

- Describing what a block of code does
- Explaining complex logic
- Leaving reminders or "to-do" notes
- Temporarily disabling (debugging) lines of code
- Adding author or version information

Example (conceptually):

```
# This code calculates the area of a rectangle
```

Good comments help others understand your program without needing to read every single line.

Types of Comments in Python

Python supports two types of comments:

1. Single-line comments:

   A single-line comment begins with the hash symbol (#). Anything after # on that line is ignored by Python.

   Example:

   ```
   # This is a single-line comment
   print("Hello, World!")  # This is also a comment after a statement
   ```

   You can place a comment on its own line or after code on the same line.

2. Multi-line comments:

   Python doesn't have a special syntax for multi-line comments like some languages (e.g., /* ... */ in C). Instead, you can use triple quotes (''' or """) to write multiple lines of text that Python will ignore if they're not assigned to a variable or used as a docstring.

   Example:

   ```
   """ --------comments---------"""
   ''' ----------comments---------'''
   ```

Lesson Program: Comments in python.

Source Code:

comments.py

```
# Python Program: Comments

# ---------------- SINGLE LINE COMMENTS ----------------
# This is a single-line comment
print("Hello, World!")   # You can also write a comment after code

# ---------------- MULTI-LINE COMMENTS ----------------
'''
This is a multi-line comment.
You can write as many lines as you want here.
Python will ignore these lines.
'''

"""
Another way to write multi-line comments
is to use triple double quotes.
This is also ignored by Python.
"""

# ---------------- PRACTICAL USE ----------------
# Storing user details
name = "John"   # student's name
age = 21         # student's age

print("Name:", name)
print("Age:", age)
```

//Keywords and Identifiers:

What are Keywords?

Keywords are special reserved words in Python that have predefined meanings and purposes. They form the core vocabulary of the Python language and are used to define its syntax and structure.

For example, words like if, else, while, for, class, True, and False are keywords. Each of these has a special meaning and function in the Python language.

For instance:

- if → used for conditional statements
- for → used for loops
- def → used to define a function
- class → used to define a class
- True and False → represent boolean values

Python keywords are case-sensitive, so True and true are not the same — True is valid, but true will cause an error. You can view all the keywords in Python using the built-in keyword module. This helps check which words are reserved and cannot be used as variable names.

Tip: You can check all keywords by writing import keyword and then print(keyword.kwlist) in your Python shell.

What are Identifiers?

Identifiers are names used to identify variables, functions, classes, modules, or other objects in Python.

For example:

- age = 20 → here, age is an identifier.
- def greet(): → here, greet is an identifier (the name of the function).

Identifiers make it easy for you (and Python) to refer to data or functions by name.

Rules for Writing Identifiers

To ensure your names are valid, Python follows these rules for identifiers:

1. Allowed Characters:
   - Can contain letters (A–Z, a–z), digits (0–9), and underscores (_)
   - Example: name, student_1, _score
2. Cannot Start with a Digit:
   - ❌ Invalid: 2name, 123abc
   - ✅ Valid: name2, a123
3. Cannot Use Keywords:
   - You cannot name something if, class, or def because they are reserved words.
4. Case Sensitive:
   - city and City are two different identifiers.

5. No Special Characters:
   - Characters like @, #, -, $ are not allowed.
   - Example: ❌ my-name is invalid.
6. Good Practice (PEP 8):
   - Use lowercase letters for variable names (total_marks)
   - Use CapitalizedWords for class names (StudentRecord)
   - Use UPPERCASE for constants (PI = 3.14)

Tip: Checking Identifiers Python provides a simple method called .isidentifier() to check if a string is a valid identifier. Similarly, the keyword module helps to verify if a word is a Python keyword or not.

Lesson Program: Keywords and Identifiers in python.

Source Code:

keywordsAndIdentifiers.py

```
# Python Keywords and Identifiers

import keyword   # module to check Python keywords

# ---------------- KEYWORDS ----------------
print("=== Keywords in Python ===")
print("Some keywords are:", keyword.kwlist[:10])
print("Total number of keywords:", len(keyword.kwlist), "\n")

# ---------------- IDENTIFIERS ----------------
print("=== Identifiers in Python ===")

name = "Alice"
age1 = 21
_student = "John"
print("Valid identifiers ->", name, age1, _student)

city = "Mumbai"
City = "Delhi"
print("\ncity:", city)
print("City:", City)

print("\nCheck if a word is identifier or keyword:")
words = ["name", "2abc", "class", "value_1"]
for w in words:
    print(w, "-> isidentifier?", w.isidentifier(), "| iskeyword?", keyword.iskeyword(w))
```

//Variables and Constants:

What is a Variable?

A variable in Python is like a container or storage box used to hold data. It stores a value that can be used and changed later in the program.

In simple terms — a variable gives a name to a value in your computer's memory so that you can reuse it easily.

Example idea:

Think of `x = 10` as naming the number 10 as "x".

Characteristics of Variables

1. Dynamic Typing (No Need for Declaration):

   In Python, you don't need to declare the type of variable before using it. When you assign a value, Python automatically decides its type.

   Example idea:

   If you write `age = 21`, Python knows age is an integer.

   Later, you can even change it to `age = "twenty-one"` — Python allows that.
2. Reassignment:

   You can assign a new value to an existing variable anytime. The old value is replaced.
3. Case Sensitivity:

   Variable names are case-sensitive — meaning `age`, `Age`, and `AGE` are three different variables.
4. Naming Rules:
   - The name must start with a letter (A–Z or a–z) or an underscore (_).
   - It can contain letters, digits, and underscores.
   - It cannot start with a number.
   - No special characters like @, #, $, -, etc.
   - It cannot use reserved keywords (like `if`, `while`, `class`, etc.).
5. Good Practice in Naming:
   - Use descriptive names that tell what the data represents.

     Example: `studentName` is better than `sn`.
   - Use `camalCase` for variable names in Python.
6. Assigning Values:
   - You can assign a value using = operator.
     - Example idea: `x = 10`
   - You can assign multiple values in one line:

     `x, y, z = 10, 20, 30`
   - Or assign one value to multiple variables:

     `a = b = c = "Python"`

7. Memory Concept (Behind the Scenes):

When you assign a variable, Python creates an object in memory and the variable name becomes a reference to that object.

Lesson Program: Variables in Python.

Source Code:

variables.py

```python
# Python Variables

# Variables store data values
name = "Alice"
age = 21
height = 5.4

print("Name:", name)
print("Age:", age)
print("Height:", height)

# Variables can be reassigned
age = 22
print("\nAfter reassignment, Age:", age)

# Valid variable name example
_valid_name = "This is valid"
print("\nValid variable name example:", _valid_name)

# Variable with underscore
my_name = "Alice Smith"
print("Variable with underscore:", my_name)

# Case-sensitive variables
city = "Mumbai"
City = "Delhi"
print("\ncity:", city)
print("City:", City)

# Multiple variables declared in one line
x, y, z = 10, 20, 30
print("\nMultiple variables -> x:", x, " y:", y, " z:", z)

# Same value to multiple variables
a = b = c = "Python"
print("Same value assigned -> a:", a, " b:", b, " c:", c)
```

What is a Constant?

A constant is a value that is not supposed to change during the execution of a program. Constants are used when a value is fixed and meaningful, such as mathematical values (PI = 3.14159) or configuration settings (MAX_USERS = 100).

In other programming languages (like C or Java), there are specific keywords like const or final to define constants but in Python, there is no built-in way to make a variable truly constant.

How Constants Work in Python?

Since Python does not have a special constant type:

- We use naming conventions to show that a variable should be treated as a constant.
- The convention is to write the constant name in all uppercase letters, often with underscores between words.

  Example:

```
PI = 3.14159
GRAVITY = 9.8
MAX_SPEED = 120
```

These are not enforced by Python, but by programmer discipline. If someone changes a constant's value later, Python will not stop them but it is considered bad practice.

Why Use Constants?

Constants make your code:

1. More readable — You instantly understand what the value represents.
2. Easier to maintain — If the value ever needs to change, you can modify it in one place.
3. Less error-prone — Prevents accidental changes to important values.

Lesson Program: Constants in Python.

Source Code:

constants.py

```python
# Python Program: Constants

# Variables
name = "Alice"
age = 21
height = 5.6

print("Variable - Name:", name)
print("Variable - Age:", age)
print("Variable - Height:", height)

# Constants (using naming convention)
PI = 3.14159
GRAVITY = 9.8
APP_NAME = "MyApp"

print("\nConstant - PI:", PI)
print("Constant - GRAVITY:", GRAVITY)
print("Constant - APP_NAME:", APP_NAME)

# Modifying constants (not recommended)
PI = 3.14
print("\n(After modifying PI) PI:", PI, "-> Not recommended!")
```

//Data Types in Python:

What Are Data Types?

Every value in Python has a data type. A data type defines what kind of value a variable holds and what operations can be performed on it.

For example:

- Numbers can be added or subtracted
- Strings can be joined
- Lists can be iterated through

Python automatically decides the data type of a variable when you assign a value — you don't have to declare it explicitly.

```
x = 5          # Integer
y = "Hello"    # String
z = 3.14       # Float
```

This feature is called dynamic typing, meaning Python figures out the type during runtime.

Why Are Data Types Important?

Data types:

1. Help Python understand how to store and manage values in memory.
2. Allow the interpreter to know what operations are valid for each type.
3. Make your programs more efficient and less error-prone.

Categories of Data Types.

Python provides several built-in data types, which can be grouped into the following categories:

Category	Data Types
Numeric Types	int, float, complex
Sequence Types	str, list, tuple
Set Type	set, frozenset
Mapping Type	dict
Boolean Type	bool
None Type	NoneType

Each data type behaves differently and serves a specific purpose in Python programming.

Type Checking

You can check the data type of any variable using the built-in type() function:

```
type(10)        # <class 'int'>
type("Hello")   # <class 'str'>
```

Python also allows you to convert one data type to another (called *type casting*), which you'll learn later.

Memory and Storage

Every data type requires some amount of memory. For example, integers and floats take up more memory than Booleans or None.

You can check the memory used by a variable with the sys.getsizeof() function.

Summary

- Python determines data types automatically (dynamic typing).
- Different data types store different kinds of information.
- You can check and compare types using built-in functions.
- Knowing data types is fundamental to writing correct Python programs.

Lesson Program: Data Types in Python.

Source Code:

dataTypes.py

```python
# Python Program: DataTypes
import sys

# Basic Data Types
integerVar = 10
floatVar = 3.14
complexVar = 2 + 5j
stringVar = "Hello Python"
boolVar = True
noneVar = None
listVar = [1, 2, 3]
tupleVar = (1, 2, 3)
setVar = {1, 2, 3}
dictVar = {"one": 1, "two": 2}

print("Integer:", type(integerVar))
print("Float:", type(floatVar))
print("Complex:", type(complexVar))
print("String:", type(stringVar))
print("Boolean:", type(boolVar))
print("None:", type(noneVar))
print("List:", type(listVar))
print("Tuple:", type(tupleVar))
print("Set:", type(setVar))
print("Dictionary:", type(dictVar))

# Storage Information
print("\n----- Data Type Sizes -----")
print("Boolean:", sys.getsizeof(boolVar), "bytes")
print("Integer:", sys.getsizeof(integerVar), "bytes")
print("Float:", sys.getsizeof(floatVar), "bytes")
print("Complex:", sys.getsizeof(complexVar), "bytes")
print("String:", sys.getsizeof(stringVar), "bytes")
print("None:", sys.getsizeof(noneVar), "bytes")
```

Numeric Data Types in Python:

What Are Numeric Data Types?

Numeric data types are used to represent numbers in Python. Python provides three built-in numeric types:

1. int – Integers (whole numbers)
2. float – Floating-point numbers (decimal values)
3. complex – Complex numbers (with real and imaginary parts)

These types allow Python to perform mathematical operations and handle both small and very large values efficiently.

1. Integer (int)

- Represents whole numbers — both positive and negative.
- Example: -5, 0, 42, 1000
- There's no fixed limit on the size of an integer (unlike other languages such as C or Java).
- Python automatically converts large integers into long integers internally.
- You can perform all arithmetic operations such as addition, subtraction, multiplication, etc.

Points to remember:

- Integers don't have decimal parts.
- You can convert integers to floats or strings using float() or str().
- The function sys.getsizeof() gives the memory usage of an integer.

2. Floating-Point (float)

- Represents real numbers with a decimal point.
- Example: 3.14, 0.99, -25.6
- Internally, floats are stored in double-precision (64-bit) format.
- They can represent very small or very large numbers using scientific notation, e.g., 2.5e3 equals 2500.0.

Points to remember:

- Converting a float to an integer removes the decimal part.
- Floats can lose precision during calculations (common in all programming languages).
- sys.float_info.max gives the maximum possible float value.

3. Complex Numbers (complex)

- Used to represent numbers with a real and an imaginary part.
- Written in the form a + bj, where a is the real part and b is the imaginary part.
- Example: 3 + 4j, 2 - 5j
- You can create them directly or by using the complex() constructor.

Points to remember:

- The real and imag attributes access the real and imaginary parts respectively.

- You can perform arithmetic operations such as addition, subtraction, and multiplication on complex numbers.
- Complex numbers are very useful in mathematics, scientific computing, and signal processing.

Checking and Conversion

You can check the type of a number using:

```
type(x)
```

You can convert between numeric types using:

```
int(), float(), complex()
```

Summary

Type	Description	Example	Notes
int	Whole numbers	10, -3, 0	No decimal part
float	Decimal numbers	3.14, -0.5	Double-precision
complex	Real + imaginary	2 + 3j	Used in scientific fields

Lesson Program: Numeric Data type in Python.

Source Code:

integerDT.py

```python
# Python Program: IntegerDataType
import sys

integerNumber = 100
print("Integer value:", integerNumber)
print("Type:", type(integerNumber))
print("Size:", sys.getsizeof(integerNumber), "bytes")

bigInteger = 10**100
print("Big integer:", bigInteger)
print("Type:", type(bigInteger))
```

floatDT.py

```python
# Python Program: FloatDataType
import sys

floatNumber = 12.75
print("Float value:", floatNumber)
print("Type:", type(floatNumber))
print("Size:", sys.getsizeof(floatNumber), "bytes")
print("Max float value:", sys.float_info.max)
```

complexDT.py

```python
# Python Program: ComplexDataType
import sys

complexNumber = 3 + 4j
print("Complex value:", complexNumber)
print("Type:", type(complexNumber))
print("Size:", sys.getsizeof(complexNumber), "bytes")

complexFromInt = complex(5)
complexFromFloat = complex(2.5)
print("Complex from int:", complexFromInt)
print("Complex from float:", complexFromFloat)
```

Strings Data Type in Python:

What Is a String?

A string in Python is a sequence of characters enclosed within single quotes ' ', double quotes " ", or triple quotes ''' ''' / """ """. It can contain letters, numbers, symbols, and even spaces.

Example:

"Hello", 'Python123', """This is a string"""

In simple words — a string is used to store and manipulate text in Python.

Characteristics of Strings

1. Strings are Immutable:
   Once created, the characters inside a string cannot be changed.
   Example:

```
text = "Hello"
# text[0] = "J" ❌ -> Error
```

2. Strings Are Indexed and Ordered:
   Every character in a string has a position (index), starting from 0.
   You can access individual characters using the index -

```
text = "Python"
print(text[0])   # P
print(text[-1])  # n (last character)
```

3. Strings Can Be Multiline:
   By using triple quotes (' ' ' or " " "), you can create a string that spans multiple lines.
4. Strings Support Concatenation and Repetition:
   You can combine strings with + and repeat them using *.

```
"Hello" + " Python"   # Concatenation
"Hi! " * 3            # Repetition
```

5. Strings Can Contain Escape Sequences:
   Use a backslash (\) for special formatting -
   - \n – newline
   - \t – tab
   - \' or \" – quotes inside strings
6. Strings Support Many Built-in Methods:
   Python provides powerful methods for string manipulation –
   text.lower(), text.upper(), text.replace(), text.find(), len(text)
7. Type Casting:
   You can convert other data types (like int or float) into strings using the str() function.
8. Memory and Storage:
   Each string takes memory proportional to its length.
   Use sys.getsizeof() to check its size in bytes.

Lesson Program: String Data type in Python.

Source Code:

stringDT.py

```python
# Python Program: StringDataType
import sys

stringText = "Hello, Python!"
print("Single-line string:", stringText)

multiLineString = """This is a
multi-line string example.
It can span multiple lines."""
print("\nMulti-line string:\n", multiLineString)

userString = input("\nEnter a string: ")
print("You entered:", userString)

stringFromInt = str(123)
stringFromFloat = str(45.67)
print("\nString from int:", stringFromInt)
print("String from float:", stringFromFloat)

print("\nSize of stringText:", sys.getsizeof(stringText), "bytes")
print("Size of multiLineString:", sys.getsizeof(multiLineString), "bytes")
```

Boolean Data Type in Python:

What is a Boolean?

A Boolean in Python is a data type that represents truth values that is, whether something is True or False. It is named after George Boole, the mathematician who introduced Boolean algebra — the foundation of logical operations in computers.

Booleans are used to make decisions in programs, typically through conditions, comparisons, and control statements like if, while, etc.

Boolean Values:

Python provides two Boolean constants:

- True
- False

These are written with an uppercase first letter — lowercase versions (true, false) will cause errors.
Example:

```
x = True
y = False
```

Boolean as a Result of Expressions:

Booleans often result from comparison or logical operations:

```
5 > 3        # True
10 == 5      # False
not True     # False
```

They are the foundation of decision-making in Python.

Numeric Representation:

In Python:

- True is internally represented as 1
- False is internally represented as 0

Hence, you can even perform arithmetic with them:

```
True + True   # 2
False + True  # 1
```

Type Casting to Boolean:

Any value can be converted to a boolean using the built-in bool() function.

- bool(0), bool(0.0), bool(""), bool([]), bool(None) → False
- Any non-zero number, non-empty string, or non-empty collection → True

This is widely used for checking emptiness or existence in Python programs.

Usage in Conditions:

Boolean expressions are mainly used in conditional statements:

```
is_logged_in = True
if is_logged_in:
    print("Welcome back!")
else:
    print("Please log in.")
```

Memory and Storage

Booleans are stored as small integer objects in memory, taking very little space (usually 28 bytes in Python).

Points to Remember

- Boolean values are only `True` or `False`.
- They are case-sensitive (capital `T` and `F` are required).
- Booleans can be results of comparisons and logical operations.
- `bool()` can convert any value to True or False.
- Internally, `True = 1` and `False = 0`.

Lesson Program: Boolean Data type in Python.

Source Code:

booleanDT.py

```python
# Python Program: Boolean (bool) Data Type

import sys

# Declaration
is_active = True
is_verified = False

# Output
print("Value of is_active:", is_active)
print("Value of is_verified:", is_verified)

# Input
user_input = input("Enter True or False: ")
user_bool = user_input == "True"
print("You entered:", user_bool)

# Type Checking
print("Type of is_active:", type(is_active))
print("Type of is_verified:", type(is_verified))

# Type Casting
print("Boolean from int(0):", bool(0))
print("Boolean from int(5):", bool(5))
print("Boolean from empty string:", bool(""))
print("Boolean from 'Hello':", bool("Hello"))
print("Boolean from float(0.0):", bool(0.0))
print("Boolean from float(3.14):", bool(3.14))

# Storage
print("Memory size of is_active:", sys.getsizeof(is_active), "bytes")
print("Memory size of is_verified:", sys.getsizeof(is_verified), "bytes")
```

None Data Type in Python

What is None?

None in Python is a special constant that represents the absence of a value or a null value. It is similar to null in other programming languages (like Java, C, or JavaScript).
It means:

"This variable exists, but it doesn't have any meaningful value yet."

Type of None:

- None belongs to the NoneType data type.
- It is a singleton object, which means there is only one instance of None in an entire Python program.
- You can check this using:

```
print(type(None))  # <class 'NoneType'>
```

Usage of None:

1. Default value for variables:
   When you want to declare a variable but don't have a value yet -

```
data = None
```

2. Default function return value:
   If a function doesn't explicitly return anything, Python automatically returns None -

```
def example():
    pass
print(example())  # Outputs: None
```

3. Used in conditional checks:
   None is often used in conditions to check if something has been assigned or not -

```
if data is None:
    print("No data available")
```

4. Placeholders in data structures:
   Used to represent missing or unknown data in lists or dictionaries -

```
user = {"name": "Alice", "age": None}
```

Comparing None:

Always use is or is not when comparing with None:

```
if value is None:
    ...
```

Don't use == or != for None comparison — is checks identity, not just equality.

Memory and Storage:

None is a single constant object in memory — it takes up very little space (usually 16 bytes) and is shared everywhere in your program.

Lesson Program: None Data type in Python.

Source Code:

noneDT.py

```python
# Python Program: None Data Type

import sys

# Declaration
emptyValue = None

# Output
print("Value of emptyValue:", emptyValue)

# Type Checking
print("Type of emptyValue:", type(emptyValue))

# Comparison
if emptyValue is None:
    print("emptyValue has no value (it's None)")

# Input Example
userInput = input("Enter something (leave blank for None): ")
if userInput == "":
    userValue = None
else:
    userValue = userInput
print("You entered:", userValue, "| Type:", type(userValue))

# Storage
print("Memory size of emptyValue:", sys.getsizeof(emptyValue), "bytes")
```

//Type Conversion in Python

What is Type Conversion?

Type conversion is the process of changing the data type of a value from one type to another. Python supports both automatic and manual conversions between compatible data types.

For example, converting an integer to a float or a string to an integer (when possible).

Types of Type Conversion

There are two types of conversions in Python:

1. Implicit Type Conversion (Automatic Conversion)

- Also called Type Coercion.
- Python automatically converts one data type into another without user intervention.
- This happens only between compatible data types.
- For example:
  - When an integer and a float are added, Python automatically converts the integer to float.
  - The goal is to avoid data loss and ensure smooth operations.

Example:

```
int_value = 10
float_value = 3.5
result = int_value + float_value   # int is converted to float automatically
```

Output will be a float, since float has higher precision.

2. Explicit Type Conversion (Manual Conversion)

- Also known as Type Casting.
- Done manually by the user using built-in functions.
- Common conversion functions:

Function	Description
int()	Converts data to integer (decimals are truncated).
float()	Converts data to float.
str()	Converts data to string.
bool()	Converts data to boolean (0, "", None → False).
complex()	Converts numbers to complex form.

Example:

```
num = "25"
converted = int(num)  # manually convert string to integer
```

If the conversion is not possible (e.g., "abc" → int), Python raises a ValueError.

Why Type Conversion is Important?

- To perform mathematical operations between different types.
- To format or display data properly.
- To store and process user input (since input is always a string).
- To avoid runtime errors caused by incompatible data types.

Key Points to Remember

- Implicit conversion is automatic, explicit is manual.
- Python only performs safe automatic conversions.
- Use conversion functions (int(), float(), etc.) carefully.
- Not all strings can be converted to numbers.
- Always check the type after conversion using type().

Lesson Program: Type Conversion in Python.

Source Code:

typeConversion.py

```python
# Python Program: Type Conversion

# ---------- Implicit Type Conversion ----------
print("----- Implicit Type Conversion -----")
numInt = 10       # integer value
numFloat = 3.5    # float value

# int is automatically converted to float during arithmetic operation
result = numInt + numFloat

print("Value of result:", result)
print("Type of result:", type(result))   # result is float


# ---------- Explicit Type Conversion ----------
print("\n----- Explicit Type Conversion -----")

# float to int (decimal part is removed)
floatNum = 9.99
intNum = int(floatNum)
print("Float:", floatNum, "| After int():", intNum)

# int to string
age = 21
ageStr = str(age)
print("Integer:", age, "| After str():", ageStr, "| Type:", type(ageStr))

# string to float
strNumber = "45.67"
floatNumber = float(strNumber)
print("String:", strNumber, "| After float():", floatNumber, "| Type:", type(floatNumber))
```

//Input & Output in Python

In Python, input and output (I/O) refer to how a program interacts with users, taking information in (input) and showing results out (output).

Output in Python

The main function for output is print().

Basic Usage:

```
print("Hello, World!")
```

It displays text, numbers, or variables on the screen.

Multiple Values:

```
print("Sum of numbers is", 10 + 20)
```

You can print multiple values separated by commas.

Parameters in print():

1. sep → Defines the separator between printed items.

```
print("A", "B", "C", sep="-")    # Output: A-B-C
```

2. end → Defines what appears at the end of the print output (default is newline \n).

```
print("Hello", end=" ")
print("World!")    # Output: Hello World!
```

Escape Sequences:

Used to format output inside strings:

Escape	Meaning	Example Output
\n	New line	Line1 Line2
\t	Tab space	Column1 Column2
\\	Backslash	\
\' or \"	Qoutation	‘ or “

Formatting Output:

Python provides several ways to format text neatly:

1. String Concatenation:

```
print("Total: " + str(45))
```

2. str.format() method:

```
print("Name: {}, Age: {}".format( *args: "John", 23))
```

3. f-Strings (modern & best):

```
name = "John"; age = 23
print(f"Name: {name}, Age: {age}")
```

Input in Python

The main function for input is input().

Basic Usage:

```
name = input("Enter your name: ")
print("Hello,", name)
```

- The prompt text appears inside parentheses.
- The return value of input() is always a string, even if the user types a number.

Converting Input:

Since input() gives a string, convert it using type casting:

```
age = int(input("Enter your age: "))
height = float(input("Enter your height in meters: "))
```

Handling Invalid Input:

To prevent errors when converting invalid data:

```
try:
    num = int(input("Enter an integer: "))
except ValueError:
    print("Invalid input! Please enter numbers only.")
```

Reading Multiple Values:

You can read several values in one line:

```
a, b = input("Enter two numbers: ").split()
a = int(a); b = int(b)
```

Handling Empty Input:

If the user presses Enter without typing anything:

```
color = input("Favorite color: ").strip()
if color == "":
    color = "Not provided"
```

Input Prompt Best Practices:

Always provide clear instructions in prompts:

```
date = input("Enter date in YYYY-MM-DD format: ")
```

Lesson Program: Input & Output n in Python.

Source Code:

input&Output.py

```python
# Python Program: Input and Output

# ----- Basic print() -----
print("Hello, World!")
print("Numbers:", 10, 20, 30)    # multiple values
print()  # blank line

# ----- print() with sep and end -----
print("A", "B", "C", sep="-")    # custom separator
print("Line without newline ->", end=" ")
print("continues same line")

# ----- Basic input() -----
name = input("\nEnter your name: ")
print("Hi,", name + "!")

# ----- Type casting -----
age = int(input("Enter your age: "))      # string → int
height = float(input("Enter height in meters: "))  # string → float
print("Age:", age, "| Height:", height)

# ----- Multiple inputs -----
a, b = input("Enter two integers separated by space: ").split()
a, b = int(a), int(b)
print("Sum =", a + b)

# ----- Handling empty input -----
color = input("Favorite color (press Enter to skip): ").strip()
if color == "":
    color = "Not provided"
print("Color:", color)

# ----- Formatted output -----
item, price, qty = "Book", 120.5, 2
print(f"Using f-string -> {item}, {price}, {qty}, Total = {price * qty:.2f}")

# ----- Escape sequences -----
print("Newline ->\\n\nTab ->\\t\tExample\tTab")
```

//Python Basics Summary

- Python syntax is clean and depends on **indentation** instead of braces {}. Consistent spacing (usually 4 spaces) defines code blocks like loops, functions, and conditionals.
- **Keywords** are reserved words that have special meaning in Python (like if, else, while, def, True, None). They cannot be used as variable names.
- **Identifiers** are names used for variables, functions, and classes. They must start with a letter or underscore and can't include special characters or spaces.
- **Comments** make code readable — single-line comments use #, and multi-line comments can be written using triple quotes (''' ... ''' or """ ... """).
- **Variables** store data values. Python is dynamically typed, so you don't need to declare types — for example, x = 10 or name = "John".
- **Constants** don't have built-in enforcement in Python but are written in uppercase (like PI = 3.14159) by convention to show they shouldn't be changed.
- **Data types** define the kind of data stored:
  - int for whole numbers (10)
  - float for decimals (3.14)
  - str for text ("Hello")
  - bool for logical values (True, False)
  - None represents the absence of a value or "no data."
- **Type conversion** changes one data type to another:
  - **Implicit** happens automatically (e.g., 3 + 2.5 becomes 5.5, a float).
  - **Explicit** uses functions like int(), float(), str(), etc., to manually convert types.
- **Input** is taken from the user using input(), which always returns a string — you must cast it to int or float when needed.
- **Output** is displayed using print(), which can show text and variables together. It supports parameters like sep (separator between items) and end (what to print at the end of the line).

In short, Python basics revolve around writing readable code with proper indentation, meaningful names, and correct handling of types and input/output.

Operators in Python

//Introduction to Operators in Python

Operators in Python are special symbols that perform actions on data. They help Python evaluate expressions, compare values, assign data, and make logical decisions. Every operator works with one or more operands (values or variables) to produce a result.

Python categorizes operators based on the type of task they perform:

- Arithmetic operators handle mathematical tasks like addition, subtraction, multiplication, division, remainder, integer division, and exponent.
- Relational (comparison) operators compare two values and return a boolean result (True/False), useful for conditions.
- Assignment operators store values in variables and also update them using combined forms like +=, -=, *=, etc.
- Logical operators (and, or, not) combine boolean expressions and help in decision-making.
- Bitwise operators work at the binary level using AND, OR, XOR, shifts, etc.
- Membership operators (in, not in) check whether a value exists inside sequences like strings, lists, or tuples.
- Identity operators (is, is not) compare the *memory location* of two objects, not their values.

Operators are essential because they allow Python to evaluate expressions, perform calculations, make comparisons, and run logic-based programs. They appear in almost every Python script, from simple math to complex decision structures.

Lesson Program: Operators in Python.

Source Code:

operators.py

```python
# Operators in Python

print("\n===== OPERATORS IN PYTHON =====")
print("Operators perform actions on values. Examples: +, -, *, /, ==, and, or, etc.\n")

# List of operator categories
print("Types of Operators:")
print("1. Arithmetic        → +  -  *  /  %  //  **")
print("2. Relational        → ==  !=  >  <  >=  <=")
print("3. Assignment        → =  +=  -=  *=  /=  %=  //=  **=")
print("4. Logical           → and  or  not")
print("5. Bitwise           → &  |  ^  ~  <<  >>")
print("6. Membership        → in  not in")
print("7. Identity          → is  is not")

# Example variables
a, b = 10, 5
x, y = True, False
text = "Python"

print("\nExample variables:")
print("a =", a, ", b =", b)
print("x =", x, ", y =", y)
print("text =", text)

# Small demonstration
print("\nQuick examples:")
print("a + b =", a + b)                 # arithmetic
print("a > b =", a > b)                 # comparison
print("x and y =", x and y)             # logical
print("'P' in text =", 'P' in text)     # membership
print("a is b =", a is b)               # identity
```

//Arithmetic Operators

Arithmetic operators are used in Python to perform mathematical calculations on numbers. These operators work with integers, floats, and even complex numbers. They are among the most frequently used tools in programming because every real-world task billing, statistics, measurements, loops, and logic uses some form of arithmetic.

The addition operator `(+)` combines two values, such as adding prices or scores. The subtraction operator `(-)` helps in finding differences—for example, remaining balance or distance left. Multiplication `(*)` is used when you need repeated addition, like calculating total cost = price × quantity. Division `(/)` always returns a decimal value, making it helpful for average calculations and ratios.

Floor division `(//)` removes the fractional part and gives only the whole number result—for example, how many full groups can be made from a total number of items. The modulus operator `(%)` returns the remainder and is widely used in tasks like checking if a number is even `(num % 2 == 0)`. Finally, the exponent operator `(**)` is used to compute powers, such as squaring a number `(n**2)` or calculating exponential growth.

Together, these operators make it easy to build logical, mathematical, and algorithmic solutions in Python.

Lesson Program: Arithmetic Operators in Python.

Source Code:

arithmeticOperators.py

```python
# Arithmetic Operators in Python

print("\n===== ARITHMETIC OPERATORS IN PYTHON =====")

# Example values
a, b = 15, 4
print("Using a =", a, "and b =", b, "\n")

# Basic arithmetic operations
print("a + b =", a + b)      # addition
print("a - b =", a - b)      # subtraction
print("a * b =", a * b)      # multiplication
print("a / b =", a / b)      # division (float)
print("a // b =", a // b)    # floor division
print("a % b =", a % b)      # remainder
print("a ** b =", a ** b)    # exponent

# User input demonstration
num1 = int(input("\nEnter first number: "))
num2 = int(input("Enter second number: "))

print("Addition:", num1 + num2)
print("Subtraction:", num1 - num2)
print("Multiplication:", num1 * num2)
print("Division:", num1 / num2)
print("Floor Division:", num1 // num2)
print("Modulus:", num1 % num2)
print("Exponent:", num1 ** num2)
```

// Relational (Comparison) Operators in Python

Relational operators are used to compare two values and decide the relationship between them. These comparisons always evaluate to a Boolean result — either True or False. They are an essential part of decision-making in programming because they help control the flow of logic, especially inside if, elif, and while statements.

The **equal to** (==) operator checks whether two values are the same, commonly used to compare user input or verify a password. The **not equal to** (!=) operator checks if two values differ—useful when validating that a value should not match something.

The **greater than** (>) and **less than** (<) operators compare numeric values, such as checking if a score crosses a threshold or if temperature drops below a safe level. Similarly, **greater than or equal to** (>=) and **less than or equal to** (<=) combine equality with comparison, often used for range checks—for example, validating if a number lies between certain limits.

These operators also work on strings using alphabetical order (lexicographical comparison). For example, "cat" > "bat" is True because 'c' comes after 'b' in the alphabet. Overall, relational operators help your program "make decisions" by comparing user inputs, numeric values, strings, or even object identities.

Lesson Program: Relational Operators in Python.

Source Code:

relationalOperators.py

```python
# Relational (Comparison) Operators in Python

print("\n===== RELATIONAL OPERATORS IN PYTHON =====")

# Example values
a, b = 10, 20
print("Using a =", a, "and b =", b, "\n")

# Basic comparisons
print("a == b ->", a == b)   # equal to
print("a != b ->", a != b)   # not equal
print("a > b  ->", a > b)    # greater than
print("a < b  ->", a < b)    # less than
print("a >= b ->", a >= b)   # greater or equal
print("a <= b ->", a <= b)   # less or equal

# User comparison example
x = int(input("\nEnter first number: "))
y = int(input("Enter second number: "))

print(x, "==", y, "->", x == y)
print(x, "!=", y, "->", x != y)
print(x, ">", y, " ->", x > y)
print(x, "<", y, " ->", x < y)
print(x, ">=", y, "->", x >= y)
print(x, "<=", y, "->", x <= y)

# String comparison demo
print("\nString comparison examples:")
print("'apple' == 'apple' ->", "apple" == "apple")
print("'apple' < 'banana' ->", "apple" < "banana")
print("'cat' > 'bat' ->", "cat" > "bat")
```

// Assignment Operators in Python

Assignment operators are used to store values in variables and update those values efficiently. The simplest form is the **= operator**, which assigns a value to a variable. For example, score = 50 stores the number 50 in score.

Python also provides **compound assignment operators**, which combine an operation with assignment in a shorter form. For example, writing count += 1 means "increase count by 1," which is cleaner than writing count = count + 1. These operators exist for addition, subtraction, multiplication, division, modulus, exponent, and floor division.

There are also **bitwise assignment operators**, used mostly in low-level or performance-oriented tasks. They work on the binary representation of numbers. For example, x &= 1 performs a bitwise AND with 1 and updates x with the result.

Assignment operators help make code shorter, more readable, and more efficient by reducing repetition—for instance, updating running totals, adjusting values in loops, or modifying flags. They work only on variables (not literal values) and update the existing variable in place.

Lesson Program: Assignment Operators in Python.

Source Code:

assignmentOperators.py

```python
# Assignment Operators in Python

print("\n===== ASSIGNMENT OPERATORS IN PYTHON =====")

# Simple assignment
a = 10
print("Initial a =", a)

# Arithmetic assignment operators
a += 5   # add and assign
a -= 3   # subtract and assign
a *= 2   # multiply and assign
a /= 4   # divide and assign
a %= 3   # modulus assign
print("After various arithmetic assignments, a =", a)

# Floor division and exponent
a = 10
a //= 3
a **= 2
print("After floor division and exponent assignments, a =", a)

# Bitwise assignment operators
a = 5
a &= 3
a |= 2
a ^= 1
a <<= 1
a >>= 2
print("Final value after bitwise assignments, a =", a)

# User input example
x = int(input("\nEnter a number: "))
x += 10
x *= 2
x //= 5
print("Final x after assignment operations =", x)
```